

1 Code Synopsis

The Vulkan GLSL code implements a GPU-resident particle collision detection system referred to as **graphics Particle Collision Detection (gPCD)**. The method uses a hybrid graphics and compute pipeline in which the graphics pipeline performs broad-phase spatial mapping while the compute pipeline performs narrow-phase collision verification. The entire collision structure remains resident in GPU memory between frames, minimizing CPU involvement and avoiding costly memory transfers.

The system is based on a uniform three-dimensional cell domain in which each cell has unit side length and its center located at positive integer coordinates. Particle positions are represented continuously in space, while spatial indexing is performed by rounding particle corner coordinates to the nearest cell center. The collision domain is flattened into a one-dimensional array using the mapping

$$I = x + W(y + Hz)$$

where W and H are the domain dimensions and each cell stores a bounded list of particle indices occupying that location.

Each particle is represented by a structured `Particle` object stored in a Vulkan SSBO using `std430` layout. The structure contains position buffers, velocity data, thermodynamic parameters, corner-cell linkage information, and collision status data. Careful attention has been given to memory alignment and layout consistency between GLSL and C++ representations. The final particle structure occupies 304 bytes and is aligned to 16-byte boundaries for efficient GPU access.

The broad-phase stage is executed in the graphics pipeline vertex shader. Each vertex invocation corresponds to one particle. For every particle, the shader computes the eight corner locations of the particle bounding volume relative to the uniform cell grid. Duplicate corner-cell assignments are eliminated during assignment rather than through a later cleanup pass, reducing unnecessary memory writes and minimizing redundant broad-phase occupancy insertions.

For each unique corner cell, the shader atomically inserts the particle index into the cell occupancy structure using a per-cell atomic counter. Particle index zero is intentionally reserved as a null or sentinel value so that zero-filled GPU buffers naturally represent empty cell entries. This permits efficient resetting of the collision structures using `vkCmdFillBuffer(...,0)` without requiring special invalid index markers.

The narrow-phase stage is executed in a compute shader during the subsequent frame. The compute stage operates on the previously generated broad-phase collision structure, introducing a one-frame latency while avoiding immediate graphics-to-compute synchronization stalls within the same frame. At common rendering frequencies such as 60 Hz or greater, this latency is approximately

$$\Delta t \approx \frac{1}{60} \approx 16.7 \text{ ms}$$

and is generally negligible for visualization and verification workloads.

During narrow-phase processing, each compute invocation examines the candidate particles contained within the corner cells associated with the active particle. Exact collision checks are then performed using geometric sphere-overlap testing based on particle radii and center distances. Collision flags are updated directly within GPU memory.

Performance testing indicated that the dominant scaling factor is local particle density per cell rather than total particle count alone. Empirical testing showed that approximately eight particles per cell produced optimal performance, balancing broad-phase memory efficiency against narrow-phase candidate counts. Since the number of cells may be expanded independently of particle count, the method can maintain near-optimal occupancy by increasing spatial resolution rather than increasing per-cell density.

The resulting system forms a GPU-driven asynchronous collision detection architecture in which broad-phase construction, narrow-phase verification, and rendering are pipelined across frames to maximize throughput while minimizing synchronization overhead.

1.1 Future Evaluation Areas

The following components will be evaluated and expanded individually in future revisions:

- Spatial indexing strategy
- Atomic insertion performance
- Occupancy scaling
- Duplicate pair suppression
- Memory alignment and SSBO layout
- Synchronization strategy
- Compute-stage divergence
- Cell topology and dimensional scaling
- GPU occupancy and throughput
- Collision verification methodology

1.2 Comparison to Other Collision Detection Methods

The current gPCD implementation differs substantially from conventional collision detection approaches in both execution model and architectural design. Traditional collision detection systems are commonly divided into broad-phase and narrow-phase stages, but most existing methods perform these operations either entirely on the CPU or entirely within compute-based GPU kernels. In contrast, gPCD distributes the workload across both the graphics and compute pipelines while maintaining the collision structures entirely within GPU memory.

Conventional broad-phase methods typically include:

- Uniform spatial grids
- Cell-linked lists
- Bounding volume hierarchies (BVH)
- Octrees and kd-trees
- Sweep-and-prune methods
- Spatial hashing

Many CPU-based approaches rely heavily on pointer traversal, recursive tree operations, or dynamically allocated spatial structures. While effective for moderate particle counts, such methods often become increasingly memory-latency bound and difficult to parallelize efficiently at very large scales.

GPU collision systems commonly employ compute shaders or CUDA kernels to construct spatial grids and perform broad-phase searches. These methods typically use:

- Parallel radix sorting
- Morton-code spatial ordering
- Hash-table construction
- Prefix-sum compaction
- Neighbor-list generation

While highly parallel, many of these approaches require multiple synchronization stages, sorting passes, or intermediate memory transformations.

The current gPCD implementation instead performs broad-phase occupancy generation directly within the graphics pipeline vertex stage. Each particle invocation computes its associated corner-cell occupancy locations and atomically inserts itself into GPU-resident cell lists. This eliminates the need for:

- Global particle sorting

- Recursive tree traversal
- CPU-side spatial reconstruction
- Prefix-sum compaction passes
- Host-device collision transfers

The resulting structure behaves similarly to a GPU-resident cell-linked list, but its construction is driven by graphics-pipeline parallelism rather than a dedicated compute-grid construction pass.

Unlike hierarchical spatial structures such as BVHs or octrees, the current implementation uses a uniform cell topology. This simplifies indexing and permits direct constant-time cell access. The tradeoff is that performance becomes more sensitive to local particle density. Preliminary testing indicates that maintaining approximately eight particles per cell provides favorable performance characteristics.

The system also differs from many traditional methods in its execution ordering. Rather than constructing and consuming the collision structure within the same frame, the broad-phase occupancy structure generated during one frame is consumed during the subsequent frame by the compute narrow-phase stage. This introduces a small one-frame latency while allowing the graphics and compute stages to remain asynchronously pipelined.

Compared to traditional CPU collision systems, the current approach offers:

- Significantly greater parallelism
- Reduced CPU involvement
- Elimination of host-device collision transfers
- Persistent GPU-resident collision structures
- Extremely high candidate-generation throughput

Compared to existing GPU compute-only approaches, the primary distinction of gPCD is the use of the graphics pipeline itself as the broad-phase collision construction mechanism.

1.3 Integrated Pipeline Considerations

A significant aspect of the current gPCD architecture is that the broad-phase collision construction and particle rendering stages are not treated as independent processes. Instead, both operations occur simultaneously within the graphics pipeline. This differs from many conventional collision detection studies in which only the isolated broad-phase algorithm is evaluated while the supporting pipeline costs are treated separately or omitted entirely.

In many published GPU collision detection systems, the reported broad-phase performance often excludes several dependent operations required by the full simulation pipeline. These may include:

- Spatial sorting
- Morton-code generation
- Prefix-sum construction
- Hash-table compaction
- Neighbor-list assembly
- CPU-GPU synchronization
- Rendering-stage reconstruction

As a result, the reported timing for the “collision detection algorithm” may represent only a subset of the total computational chain required for practical simulation and rendering.

The current gPCD implementation instead merges broad-phase collision construction directly into the rendering pipeline itself. During graphics rendering, each particle simultaneously contributes to:

- Visual rendering
- Spatial occupancy generation
- Cell-linked collision construction

Consequently, the broad-phase stage is not executed as an isolated preprocessing operation, but rather as a byproduct of the rendering pass already required for visualization. This effectively reduces two traditionally separate GPU processes into a single pipeline stage.

The effective execution chain therefore becomes:

$$\text{Broad Phase} + \text{Rendering} \rightarrow \text{Narrow Phase}$$

rather than:

$$\text{Broad Phase} \rightarrow \text{Sorting/Compaction} \rightarrow \text{Rendering} \rightarrow \text{Narrow Phase}$$

This distinction may substantially affect practical performance comparisons. A method that appears faster when evaluating only isolated broad-phase construction may require additional preprocessing and rendering stages that are not included in the reported timings. In contrast, the current gPCD implementation performs broad-phase occupancy generation concurrently with rendering, potentially reducing total pipeline cost even if the isolated broad-phase timing alone is not minimized.

This integration of rendering and broad-phase construction is believed to be one of the primary architectural advantages of the current approach and may not be fully reflected in evaluations that consider only isolated collision-detection kernels.

1.4 Memory Scaling Considerations

The current implementation uses a uniform three-dimensional cell domain whose total memory requirement scales approximately with the cube of the domain resolution. Consequently, increasing the spatial resolution of the collision field may produce substantial memory growth.

Although this cubic memory scaling is memory intensive, it does not substantially alter the computational complexity of particle placement within the collision structure itself. Each particle is still inserted through direct constant-time index evaluation and bounded occupancy insertion operations.

The particle insertion process consists primarily of:

- Corner-cell coordinate evaluation
- Direct array index flattening
- Atomic occupancy insertion

These operations remain independent of the total number of cells allocated within the domain. Increasing the spatial resolution therefore increases memory consumption far more rapidly than it increases the computational complexity of particle insertion.

Unlike hierarchical spatial structures such as BVHs, octrees, or recursive subdivision methods, the current implementation does not require:

- Tree traversal
- Recursive node subdivision
- Dynamic memory allocation
- Pointer chasing
- Neighbor hierarchy reconstruction

Cell access remains direct and constant-time regardless of domain size:

$$I = x + W(y + Hz)$$

As a result, increasing the number of cells primarily affects:

- Total GPU memory usage
- Cache locality
- Memory bandwidth requirements

rather than the asymptotic complexity of particle placement itself.

This behavior may become increasingly advantageous on modern GPUs where arithmetic throughput is extremely high relative to recursive memory-access operations. The present implementation therefore trades increased memory consumption for reduced algorithmic complexity and highly parallel constant-time spatial insertion.

2 Algorithm

The gPCD algorithm is organized as a frame-pipelined GPU process in which broad-phase collision construction and particle rendering occur simultaneously within the graphics pipeline, while narrow-phase collision verification is executed asynchronously in a compute shader.

Unlike many conventional collision detection systems, the broad-phase stage is not implemented as a separate preprocessing operation. Instead, the rendering pass itself constructs the collision occupancy structure during particle rendering. The resulting execution chain is therefore

$$\text{Broad Phase} + \text{Rendering} \rightarrow \text{Narrow Phase}$$

rather than separate broad-phase, rendering, and narrow-phase stages.

The collision structures remain resident in GPU memory between frames. The broad-phase occupancy structure generated during frame n is consumed by the narrow-phase stage during frame $n + 1$, introducing a one-frame latency while avoiding immediate graphics-to-compute synchronization stalls.

2.1 Frame Execution Pipeline

The execution order for frame n is

$$\text{Compute Narrow Phase}_{n-1} \rightarrow \text{Clear Collision Structures}_n \rightarrow \text{Graphics Broad Phase} + \text{Render}_n$$

where the broad-phase occupancy field generated during frame n becomes the narrow-phase input for frame $n + 1$.

2.2 Data Structures

Let

- P denote the particle array
- C denote the cell occupancy array
- L denote the per-cell occupancy counter array

where

- P_i is particle i
- C_j is the occupancy list for cell j
- L_j is the number of particles inserted into cell j

Particle index 0 is intentionally reserved as a sentinel value so that zero-filled GPU buffers naturally represent empty occupancy entries.

The three-dimensional collision domain is flattened into a one-dimensional array using

$$I = x + W(y + Hz)$$

where W and H are the domain dimensions.

2.3 Graphics Broad-Phase Construction and Rendering

The graphics pipeline simultaneously performs particle rendering and broad-phase collision structure generation. Each vertex invocation corresponds to one particle and computes the spatial occupancy information associated with the particle bounding volume while also contributing to the rendered image.

For each particle, the shader computes the eight corner occupancy locations associated with the particle bounding volume relative to the uniform cell field. Duplicate corner-cell assignments are removed during assignment rather than through a later cleanup pass.

2.4 Graphics Broad-Phase Construction and Fragment Rendering

The graphics pipeline performs two coupled operations during the same draw call. In the vertex stage, each particle invocation constructs the broad-phase cell occupancy information. The vertex shader computes the occupied corner cells of each particle and inserts the particle index into the corresponding GPU-resident cell lists.

After vertex processing, the particle primitive is rasterized. The fragment shader is then invoked for the fragments generated by the rasterized particle primitive and applies the particle color. Thus, rendering occurs through the standard fragment stage, while broad-phase collision construction occurs in the vertex stage of the same graphics pass.

2.5 Compute Narrow-Phase Verification

The narrow-phase stage is executed asynchronously in a compute shader. Each compute invocation processes one particle and examines the candidate particles located within the cells associated with that particle.

2.6 Collision Verification

The current narrow-phase verification method uses geometric sphere-overlap testing:

$$\|\mathbf{x}_i - \mathbf{x}_k\|^2 \leq (r_i + r_k)^2$$

where $\mathbf{x}_i$ and $\mathbf{x}_k$ are the particle centers and r_i , r_k are the particle radii.

Algorithm 1 Graphics Broad-Phase Construction and Fragment Rendering

Require: Particle array P

Require: Cell occupancy array C

Require: Cell counter array L

Ensure: Updated broad-phase occupancy structure and rendered particle image

```
1: for all particles  $P_i$  in parallel at the vertex stage do
2:   if  $i = 0$  then
3:     continue
4:   end if
5:   Compute the eight bounding corner positions of  $P_i$ 
6:   Convert corner positions to cell coordinates
7:   Remove duplicate corner-cell assignments during insertion
8:   for all unique occupied cells  $c$  do
9:      $j \leftarrow \text{ArrayToIndex}(c)$ 
10:    if  $j$  is invalid then
11:      continue
12:    end if
13:     $s \leftarrow \text{atomicAdd}(L_j, 1)$ 
14:    if  $s < N_{\text{occ}}$  then
15:       $C_j[s] \leftarrow i$ 
16:    end if
17:  end for
18:  Emit graphics primitive for particle  $P_i$ 
19: end for
20: for all fragments generated by rasterized particle primitives do
21:   Apply particle color
22: end for
```

Algorithm 2 Compute Narrow-Phase Collision Verification

Require: Particle array P

Require: Broad-phase occupancy structure from previous frame

Ensure: Updated particle collision flags

```
1: for all particles  $P_i$  in parallel do
2:   if  $i = 0$  then
3:     continue
4:   end if
5:   Clear collision flag for  $P_i$ 
6:   for all occupied cells  $c$  associated with  $P_i$  do
7:      $j \leftarrow \text{ArrayToIndex}(c)$ 
8:     for  $s = 0$  to  $L_j - 1$  do
9:        $k \leftarrow C_j[s]$ 
10:      if  $k = 0$  or  $k = i$  then
11:        continue
12:      end if
13:      if  $\text{Contact}(P_i, P_k)$  then
14:        Mark  $P_i$  as colliding
15:      end if
16:    end for
17:  end for
18: end for
```

2.7 Memory and Scaling Characteristics

The current implementation uses a uniform three-dimensional cell field whose memory usage scales approximately with the cube of the spatial resolution. Although memory intensive, increasing the number of cells does not substantially alter the complexity of particle insertion itself.

Particle insertion consists primarily of:

- Direct coordinate evaluation
- Constant-time index flattening
- Atomic occupancy insertion

Unlike hierarchical spatial methods, the implementation does not require recursive traversal, dynamic subdivision, or pointer-based memory access. Spatial lookup therefore remains direct and constant-time regardless of total domain size.

Preliminary testing indicates that local particle density per cell is one of the dominant performance parameters. Initial performance testing suggests that approximately eight particles per cell provides favorable performance characteristics by balancing occupancy density against narrow-phase candidate counts.

2.8 Possible Improvements

A potential future improvement would be to extend the graphics-stage broad-phase method by using fragment-level parallelism more explicitly. In the current implementation, the vertex shader performs the broad-phase cell insertion, while the fragment shader is primarily used to apply particle color to rasterized fragments.

In principle, performance could improve if the graphics pipeline allowed the number of fragment invocations per particle to be specified explicitly and independently of screen-space rasterization. Such a mechanism would allow each particle to generate a fixed number of guaranteed parallel invocations for collision-related work.

This would be especially useful if the following conditions could be guaranteed:

- A fixed number of fragment invocations could be requested per particle.
- These invocations would occur even if the particle primitive is clipped, cropped, occluded, or otherwise not visible.
- The fragment invocations could be treated as a reliable hierarchical parallel work group.
- The generated invocations could safely write to collision structures independent of image visibility.

Under these conditions, the graphics pipeline could provide an additional level of structured parallelism. Each particle-level vertex invocation could generate a controlled set of fragment-level invocations, allowing collision-related work to be distributed more finely within the graphics pass.

The desired execution pattern would be

Particle Invocation $\rightarrow$ Fixed Fragment-Level Work Set $\rightarrow$ Cell-Level Collision Operations

This would create a form of nested or hierarchical GPU parallelism in which one particle invocation spawns a bounded set of lower-level invocations. Such a structure could potentially reduce per-invocation workload, improve occupancy, and allow broad-phase construction to scale more efficiently.

However, standard rasterization does not provide this guarantee. Fragment shader execution depends on screen-space coverage, clipping, viewport state, visibility, and rasterization rules. Therefore, fragment invocations cannot currently be relied upon as a deterministic collision work generator in the same way as compute shader invocations.

As a result, the current implementation correctly performs collision-structure construction in the vertex stage, where one invocation per particle can be controlled directly. Fragment processing remains useful for rendering, but future graphics or GPU programming models that expose guaranteed fragment-style work generation could improve the performance and flexibility of the gPCD method.

2.9 Novelty Statement

The primary novelty of the current gPCD method is the use of the graphics pipeline itself as the broad-phase collision construction mechanism. Rather than treating rendering and collision detection as separate GPU processes, the method merges broad-phase occupancy generation directly into the graphics rendering pass while maintaining the collision structures entirely within GPU memory.

Unlike many existing GPU collision systems that rely on compute-only spatial construction, sorting passes, prefix-sum operations, or hierarchical traversal structures, the present implementation performs direct particle-to-cell insertion during the graphics vertex stage using highly parallel rasterization-driven execution.

The method therefore introduces several distinguishing characteristics:

- Simultaneous broad-phase construction and rendering within the same graphics pass
- GPU-resident collision structures maintained across frames
- Direct constant-time spatial indexing using a uniform cell field
- Elimination of global particle sorting and recursive spatial traversal
- Frame-pipelined asynchronous graphics and compute execution
- Collision occupancy generation driven by graphics-pipeline parallelism rather than dedicated compute-grid construction

A further distinguishing aspect of the approach is that the total execution chain is considered as an integrated pipeline rather than as isolated collision kernels. Many published collision detection methods evaluate only a subset of the required computational chain while excluding supporting preprocessing operations such as sorting, compaction, reconstruction, or rendering-stage integration. In contrast, the present implementation incorporates broad-phase occupancy construction directly into the rendering process itself.

The effective execution chain is therefore

$$\text{Broad Phase} + \text{Rendering} \rightarrow \text{Narrow Phase}$$

rather than a sequence of independently executed preprocessing and rendering stages.

The method also introduces a GPU-resident frame-pipelined collision architecture in which the broad-phase occupancy field generated during one frame is consumed asynchronously by the narrow-phase stage during the subsequent frame. This reduces immediate graphics-to-compute synchronization requirements while maintaining continuous GPU execution.

Collectively, these features form a graphics-driven collision detection architecture that differs from both traditional CPU spatial structures and conventional GPU compute-only collision pipelines.